

I. Formularea Proiectului:

Proiectul a constat în crearea unei anumite figuri (personaje din desene animate, elemente de natură, vase, mașini, etc) folosind curbele Bezier de diferite grade, în planul xOy.

De dragul nostalgiei, am ales să construiesc, cu ajutorul curbelor Bezier, personajul 'Curaj', din desenul animat 'Curaj, câinele cel fricos'.

II. Rezolvarea Proiectului:

Pentru a crea această figură, a fost nevoie de anumite date de intrare și de anumiți algoritmi.

- A. Date de intrare: Datele de intrare sunt reprezentate de coordonatele anumitor puncte, care vor influența gradul și forma curbei Bezier.
- B. Algoritmi: Algoritmii folosiți în acest proiect au fost : Algoritmul lui Casteljau, Interpolarea F-Mill, Interpolarea Bezier spline cubică.

Datele de intrare, respectiv coordonatele punctelor care influențează gradul și forma curbelor Bezier au fost introduse de la tastatură, acestea fiind introduse în structuri de date, repartizate la nivel de subprograme, care sunt realizate astfel încât, figura să fie formată din mai multe minifiguri.

Personajul Curaj, fiind un personaj destul de rotund, a fost nevoie de folosirea mai multor puncte de control pentru a putea desena anumite părți ale corpului lui.

Punctele de control au fost preluate cu ajutorul unei unelte online, numita Desmos. Prin aceasta aplicație, am putut încărca imaginea după care m-am ghidat, am putut crea puncte care le puteam plasa oriunde pe graficul xOy, iar prin mai multe încercări de a trasa curba, astfel încât să coincidă cu marginea imaginii, am putut salva coordonatele punctelor care îmi vor afișa curba respectivă.

În funcție de gradul de complexitate sau de forma anumitor părți din corpul personajului, unele figuri sunt formate dintr-o singură curbă sau mai multe.

III. Prezentarea codului:

(Pentru rularea codurilor se vor apela funcțiile : **Curaj.m ;**
Reprezentare_puncte_de_control.m ; Spline.m ; F_mill.m ;
Modificare_ureche.m ; Casteljau.m)

Scriptul principal : Curaj.m

```
t = 0:0.01:1;
```

Raportul proiectului de creativitate

Oltean Costin-Petruț

```
hold on
axis ([-6 6 -6 6]);
Ureche_dreapta;
Contur_dreapta_cap;
Mana_dreapta;
Coadă;
Picior_stanga;
Picior_dreapta;
Contur_dreapta;
Bot_si_nas;
Contur_stanga_cap;
Spranceana_stanga_ureche;
Ochi_dreapta;
Spranceana_dreapta;
Ochi_stanga;
Mana_stanga;
Gura;
Pete_maini_picioare;
Interior_ochi;
Pete_corp;
Mustati;
Alunite;
title('Curaj, cainele cel fricos');
```

Acesta este codul principal care desenează personajul 'Curaj'. Acest cod este format dintr-o succesiune de subprograme care sunt apelate pentru a desena anumite părți, care în final, vor reprezenta toată figura.

Aceste subprograme sunt structurate sub forma:

```
b = [-1.1, -2.3, -2.8, -5.3, -6.1, -3.4, -3.65, -1.22, -1.27; 3.8, 6.2,
2.9, 5.2, 4.8, 2.8, 3.7, 5, 3.7];
f = ReprezentareCurbaBezier(t,b);
plot(f(1,:),f(2,:), 'k', 'LineWidth', 1.5);
```

Acesta este codul primului subprogram, denumit 'Ureche_dreapta'. Subprogramul este structurat astfel:

1. Declararea variabilei b : Variabile b reprezintă o matrice formată din 2 linii și n coloane, unde n este determinat de numărul .Prima linie reprezintă valorile de pe axa OX a punctelor, iar a doua linie reprezintă valorile de pe axa OY a punctelor.
2. Declararea variabilei f : Variabila f reprezintă variabila în care se stochează rezultatul înmulțirii dintre Polinoamele Bernstein și punctele de control.Mai precis, valorile curbei Bezier.

Raportul proiectului de creativitate
Oltean Costin-Petruț

3. Afișarea curbei : Folosind comanda `plot(f(1,:),f(2,:), 'k', 'LineWidth',1.5)`, putem afișa curba Bezier, generata de punctele de control respective.

Cu cat este nevoie de împărțirea anumitor părți în mai multe segmente, este necesară introducerea de mai multe puncte de control, respectiv, de generarea a mai multor curbe.

Un astfel de exemplu ar putea fi subprogramul `Mana_stanga.m`:

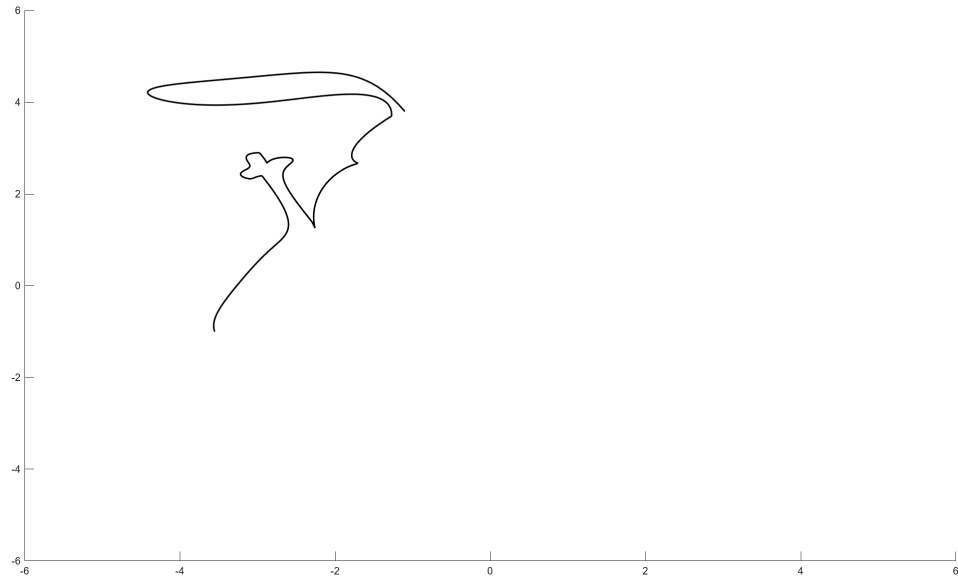
```
b = {[2.305 2.536 2.789; 2.444 2.711 2.8346], ...  
     [2.789 2.828 3.184 2.9946 ; 2.8346 2.634 2.569 2.973], ...  
     [2.9946 3.263 3.16 2.793 ; 2.973 3.107 3.477 3.184], ...  
     [2.793 2.626 2.425 2.612; 3.184 3.364 3.133 2.966], ...  
     [2.612 2.358 2.135; 2.966 2.75 2.511], ...  
    };  
for i = 1:length(b)  
    f = ReprezentareCurbaBezier(t,b{i});  
    plot(f(1,:),f(2,:), 'k', 'LineWidth',1.5);  
end
```

În acest subprogram, mâna pe care am vrut să o generez era mult prea complexă pentru a putea fi construită dintr-o singură curba Bezier, deoarece ar fi fost nevoie de mult mai multe puncte de control pentru a surprinde corect forma mainii. Astfel, am optat pentru generarea mai multor curbe Bezier, în care ultimul punct de control din prima matrice este primul punct de control pentru a doua matrice. Făcând acest lucru, oferim desenului continuitate și omitem posibilitatea de a avea spații lipsa.

În codul de mai sus am definit structura `b`, care conține toate punctele de control de care am nevoie pentru generarea mainii. Apoi, folosind un `for` loop, trecem prin fiecare element al structurii `b`, calculăm curba corespunzătoare, apoi o afișăm.

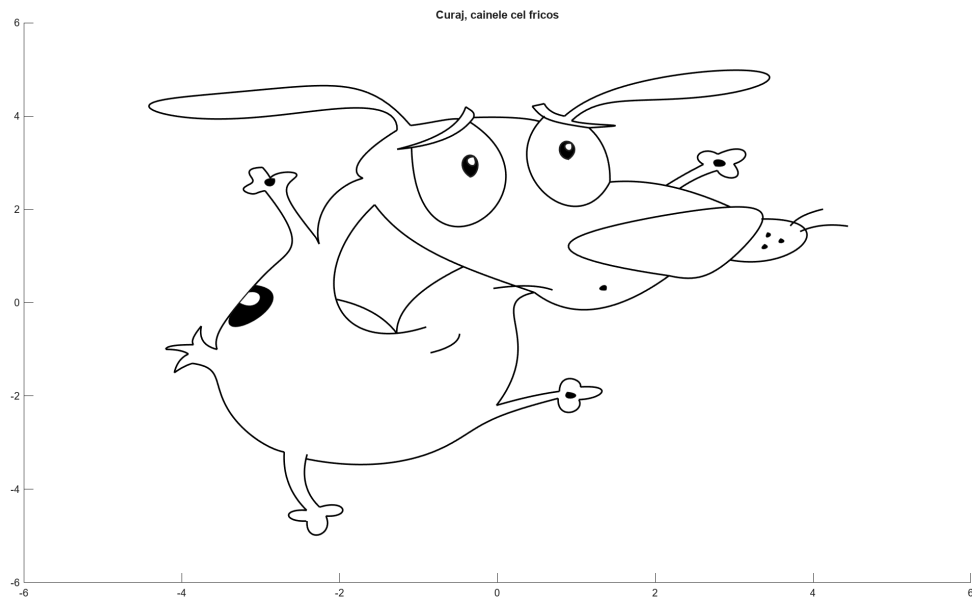
Dacă ar fi să rulăm codul principal, având doar primele 3 subprograme, figura ar arăta astfel :

Raportul proiectului de creativitate Oltean Costin-Petruț



După cum se observă, rulând doar primele 3 subprograme, obținem o parte din conturul figurii.

Dacă ar fi să rulăm fiecare subprogram, figura ar arăta astfel :



Fiecare parte din aceasta figură este realizat într-un subprogram separat.

Funcția de calculare a curbei Bezier:

Pentru a putea afișa o curba Bezier, este necesară efectuarea înmulțirii dintre polinoamele Bernstein și punctele de control. Aceasta operație este efectuată de următorul subprogram :

Raportul proiectului de creativitate

Oltean Costin-Petruț

```
function [rezultat, segmente] = ReprezentareCurbaBezier(t, puncteControl)
    n = size(puncteControl, 2) - 1;
    matriceB = zeros(length(t), n+1);
    for i = 0:n
        coefBinomial = nchoosek(n, i);
        matriceB(:, i+1) = coefBinomial .* (1-t).^(n-i) .* t.^i;
    end
    rezultat = matriceB * puncteControl';
    rezultat = rezultat';
    segmente = puncteControl(:, 1:end-1);
    segmente = reshape(segmente, 2, []);
end
```

Acest subprogram este responsabil pentru calcularea polinoamelor Bernstein de ordin n și de calcularea curbei.

În funcție de numărul punctelor de control, programul va calcula polinoamele Bernstein de ordin n .

Programul calculează ordinul polinoamelor în funcție de numărul punctelor de control, scăzând 1 din numărul acestora. Apoi, își alocă memorie pentru matricea în care vor fi stocate valorile polinoamelor. Pentru fiecare i de la 0 până la gradul polinomului, se calculează valoarea combinărilor de n luate câte i , folosind comanda `coefBinomial = nchoosek(n, i);`. După ce calculează valorile combinărilor, se calculează elementele matricii B prin comanda `matriceB(:, i+1) = coefBinomial .* (1-t).^(n-i) .* t.^i;` (Formula pentru calcularea polinoamelor Bernstein).

Dupa ieșirea din structura repetitivă, este calculată curba Bezier și este returnată, alături de punctele de control.

Reprezentarea figurei și a punctelor de control:

Una dintre cerințele proiectului a fost desenarea curbelor Bezier, alături de poligoanele lor de control:

```
figure;
t = 0:0.01:1;
title('Curaj, cainele cel fricos, alaturi de poligoanele de control');
b = {
    [-1.1, -2.3, -2.8, -5.3, -6.1, -3.4, -3.65, -1.22, -1.27; 3.8, 6.2,
    2.9, 5.2, 4.8, 2.8, 3.7, 5, 3.7],...
    [-1.27 -3 0.3 -3.4 -2 -2.3 ; 3.7 1.9 2.8 3.7 0.5 1.4],...
    [-2.3 -3.1 -2.7 -1.5 -3.6 -2.6 -2.975 ; 1.4 3.1 2.4 2.8 3.3 2.2
    2.9],...
    [-2.975 -3.4 -3.1 -2.3 -4.4 -2.6 -3.281 -2.943 ; 2.9 2.9 2.4 2.6 2.6
    2.2 2.3 2.4],...
    [-2.943 -2 -3.2 -2.7 -3.7 -3.55 ; 2.4 0.4 1.5 0.3 -0.3 -1],...
    [-3.55 -3.8 -3.75 ; -1 -0.9 -0.5],...
    [-3.75 -3.9 -3.85 ; -0.5 -0.8 -0.9],...
    [-3.85 -4.2 -4.2 ; -0.9 -0.9 -1],...
}
```

Raportul proiectului de creativitate

Oltean Costin-Petruț

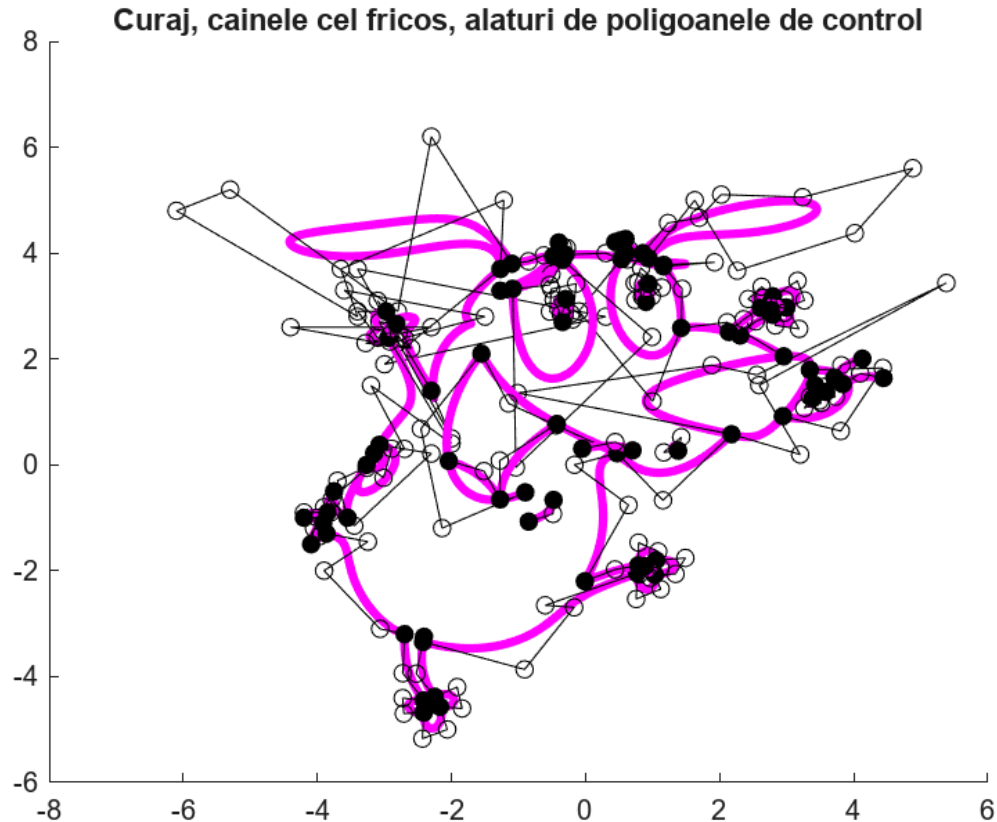
```
[-4.2 -4 -3.914 ; -1 -1 -1.1],...
[-3.914 -4.05 -4.093 ; -1.1 -1.2 -1.5],...
[-4.093 -3.968 -3.863 ; -1.5 -1.35 -1.3],...
[-3.863 -3.247 -3.9 -3.064 -2.7 ; -1.3 -1.452 -2 -3.1 -3.2],...
[-2.7 -2.726 -2.415 ; -3.2 -3.938 -4.45],...
[-2.415 -2.728 -2.711 -2.421 ; -4.45 -4.408 -4.7 -4.68],...
[-2.411 -2.434 -2.063 -2.171 ; -4.68 -5.174 -5 -4.572],...
[-2.171 -1.842 -1.916 -2.251 ; -4.572 -4.604 -4.2 -4.381],...
[-2.251 -2.525 -2.41 ; -4.381 -3.946 -3.25],...
[-2.424 -0.91 -0.17 -0.603 0.77 ; -3.346 -3.866 -2.693 -2.654
-2.05],...
[0.77 0.754 1.12 1.032 ; -2.05 -2.536 -2.348 -2.077],...
[1.032 1.341 1.488 1.052 ; -2.077 -2.061 -1.76 -1.8044],...
[1.052 1.082 0.79 0.786; -1.8044 -1.6366 -1.47 -1.9],...
[0.786 0.439 -0.011 ; -1.9 -1.978 -2.2],...
[-0.011 0.64 -0.166 0.47 ; -2.2 -0.76 0 0.22],...
[0.47 1.155 2.1754 ; 0.22 -0.67 0.578],...
[2.1754 -1.014 1.883 2.554 5.38 2.59 3.2 2.1754 ; 0.578 1.357 1.883
1.7 3.436 1.505 0.195 0.578],... %25
[2.9415 3.806 4.432 3.345 ; 0.9194 0.638 1.83 1.794],...
[2.954 2.112 1.426 ; 2.0507 2.693 2.587],...
[1.426 1.44 1.16 ; 2.587 3.326 3.75],...
[1.16 0.628 0.57 0.442 ; 3.75 3.88 3.946 4.211],...
[0.442 0.5232 0.595 ; 4.211 4.2398 4.2667],... %30
[0.595 0.653 0.8516; 4.2667 4.062 4],...
[0.8516 1.23 1.694 2.023 3.236 4.88 4.016 2.26 1.628 0.94 ; 4 4.57
4.667 5.106 5.057 5.604 4.377 3.667 5 3.9],...
[0.94 1.162 1.924 1.16 ; 3.9 3.778 3.827 3.75],...
[1.426 1 -0.1 0.6; 2.587 1.2 2.9 4],... %34
[0.53457 0.3 -0.3 ; 3.8856 4 3.97],...
[-0.3 -0.28 -0.34 -0.4 ; 3.97 4.1 4.12 4.2],...
[-0.4 -0.51 -1.27 ;4.2 3.6 3.29],...
[-1.27 -0.534 -0.3 ; 3.29 3.4 3.97],...
[-0.484 -0.623 -0.85 -1.1 ; 3.939 3.958 3.846 3.8],...
[-1.087 -1.04 0.99 -0.346 ; 3.326 -0.05 2.415 3.871],...
[2.305 2.536 2.789; 2.444 2.711 2.8346],...
[2.789 2.828 3.184 2.9946 ; 2.8346 2.634 2.569 2.973],... %40
[2.9946 3.263 3.16 2.793 ; 2.973 3.107 3.477 3.184],...
[2.793 2.626 2.425 2.612; 3.184 3.364 3.133 2.966],...
[2.612 2.358 2.135; 2.966 2.75 2.511],... %45
[-0.9 -2.14 -2.456 -1.554;-0.52 -1.196 0.68 2.101],...
[-1.554 -1.15 -0.442 0.47; 2.101 1.16 0.744 0.22],...
[-0.430 -1.28 -1.275 ; 0.773 0.08 -0.654],...
[-1.275 -1.516 -2.04 ; -0.654 -0.118 0.0728],...
[-0.478 -0.486 -0.847 ; -0.664 -0.926 -1.073],...
[-2.822 -3.075 -2.918 -2.7874 -2.822 ; 2.661 2.703 2.33 2.5035
2.661],...
[0.885 0.799 0.964 1.118 0.885 ; -1.914 -2.072 -2.1344 -1.951
-1.914],... %52
```

Raportul proiectului de creativitate

Oltean Costin-Petruț

```
[2.7614 2.96 2.969 2.645 2.7614 ; 2.9263 2.873 3.119 3.155 2.9263],...
[-0.3425 -0.194 -0.141 -0.512 -0.532 -0.3425 ; 2.7 2.747 3.424 3.323
2.894 2.7],...
[0.9 1.136 0.827 0.732 0.742 0.9; 3.078 3.35 3.721 3.434 3.114
3.078],...
[-0.297 -0.476 -0.369 -0.218 -0.297 ; 3.133 3.122 2.842 2.8575
3.133],...
[0.939 0.784 0.843 0.987 0.939 ; 3.4193 3.457 3.174 3.186 3.4193],...
[-3.073 -2.3 -3.45 -3.787 -3.265 -3.073; 0.3785 0.214 -1.15 -0.655
-0.058 0.3785],...
[-3.1526 -2.89 -3.014 -3.266; 0.2235 0.304 -0.247 -0.00127],...
[0.7 0.442 -0.05 ; 0.273 0.43 0.306],...
[3.712 3.815 4.127 ; 1.645 1.883 2.005],...
[3.836 4.095 4.443 ; 1.526 1.726 1.64],...
[3.421 3.547 3.346 3.421 ; 1.506 1.471 1.29 1.506],...
[3.589 3.729 3.49 3.589 ; 1.379 1.288 1.213 1.379],...
[3.385 3.516 3.2597 3.385 ; 1.2563 1.168 1.074 1.2563],...
[1.379 1.1655 1.427 1.379 ; 0.268 0.235 0.523 0.268],...
};
hold on
for i =1:length(b)
    f = ReprezentareCurbaBezier(t,b{i});
    plot(f(1,:),f(2,:), 'm', 'LineWidth',3);
    plot(b{i}(1,1),b{i}(2,1), 'Color','k', 'Marker','o',
'MarkerFaceColor','k');
    plot(b{i}(1,end),b{i}(2,end), 'Color','k', 'Marker','o',
'MarkerFaceColor','k');
    plot(b{i}(1,:),b{i}(2,:), 'Color','k','Marker','o');
end
```

Codul de mai sus afișează figura cu o grosime de 3 și culoarea magenta, pentru distingerea curbei față de poligoanele de control. Primul punct și ultimul punct extrem vor fi marcate cu marcatorul 'o' și umplute cu negru, iar celelalte puncte se vor distinge de acestea două, doar prin marcarea acestora cu marcatorul 'o'.



Algoritmul lui Casteljau:

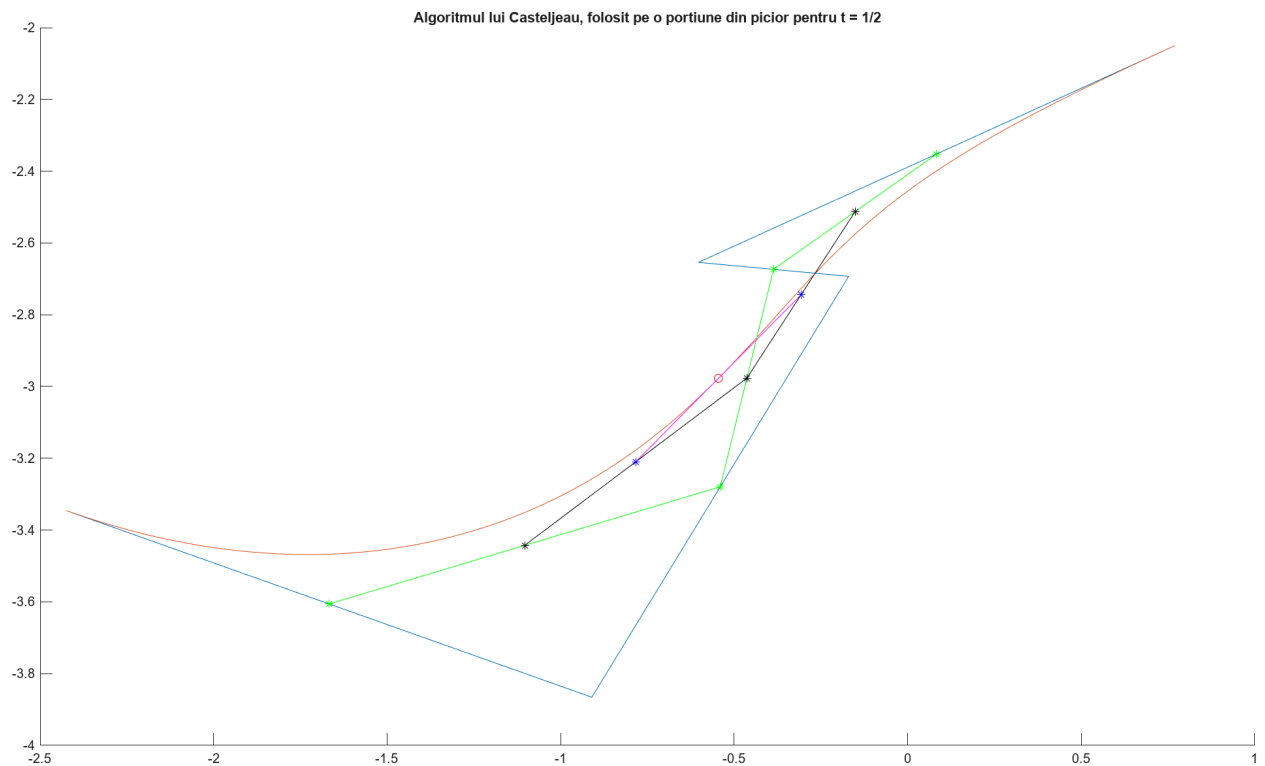
Algoritmul lui Casteljau este o metodă iterativă folosită pentru evaluarea și vizualizarea curbelor Bézier. În esență, acesta interpolează punctele de control ale unei curbe Bézier folosind ponderi calculate în funcție de parametrul t și formează noi puncte intermediare până când se ajunge la un singur punct, care reprezintă un punct de pe curba Bézier la o anumită valoare a lui t . Această metodă este eficientă și permite evaluarea curbei în mod dinamic la orice valoare a parametrului t , facilitând astfel reprezentarea grafică precisă a curbelor Bézier.

```
figure;  
title('Algoritmul lui Casteljeau, folosit pe o portiune din picior pentru t =  
1/2');  
b = [-2.424 -0.91 -0.17 -0.603 0.77; -3.346 -3.866 -2.693 -2.654 -2.05];  
t = 0:0.01:1;  
B0 = (1 - t) .^ 4;  
B1 = 4 * (1 - t) .^ 3 .* t;  
B2 = 6 * (1 - t) .^ 2 .* t .^ 2;  
B3 = 4 * (1 - t) .* t .^ 3;  
B4 = t .^ 4;  
B = [B0;B1;B2;B3;B4];  
f = b * B;  
hold on
```

Raportul proiectului de creativitate

Oltean Costin-Petruț

```
plot(b(1,:),b(2,:));
plot(f(1,:),f(2,:));
t = 1/2;
b1 = zeros(2,4) ; b2 = zeros(2,3) ; b3 = zeros(2,2) ; b4 = zeros(2,1);
for i = 1:4
    b1(:,i) = b(:,i)*(1-t) + b(:,i+1)*t;
end
for i = 1:3
    b2(:,i) = b1(:,i)*(1-t) + b1(:,i+1)*t;
end
for i = 1:2
    b3(:,i) = b2(:,i)*(1-t) + b2(:,i+1)*t;
end
b4(:,1) = b3(:,1)*(1-t) + b3(:,2)*t;
plot(b1(1,:),b1(2,:), 'g*')
plot(b1(1,:),b1(2,:), 'g-');
plot(b2(1,:),b2(2,:), 'k*');
plot(b2(1,:),b2(2,:), 'k-');
plot(b3(1,:),b3(2,:), 'm-');
plot(b3(1,:),b3(2,:), 'b*');
plot(b4(1,:),b4(2,:), 'ro');
hold off
```



Interpolarea F-Mill:

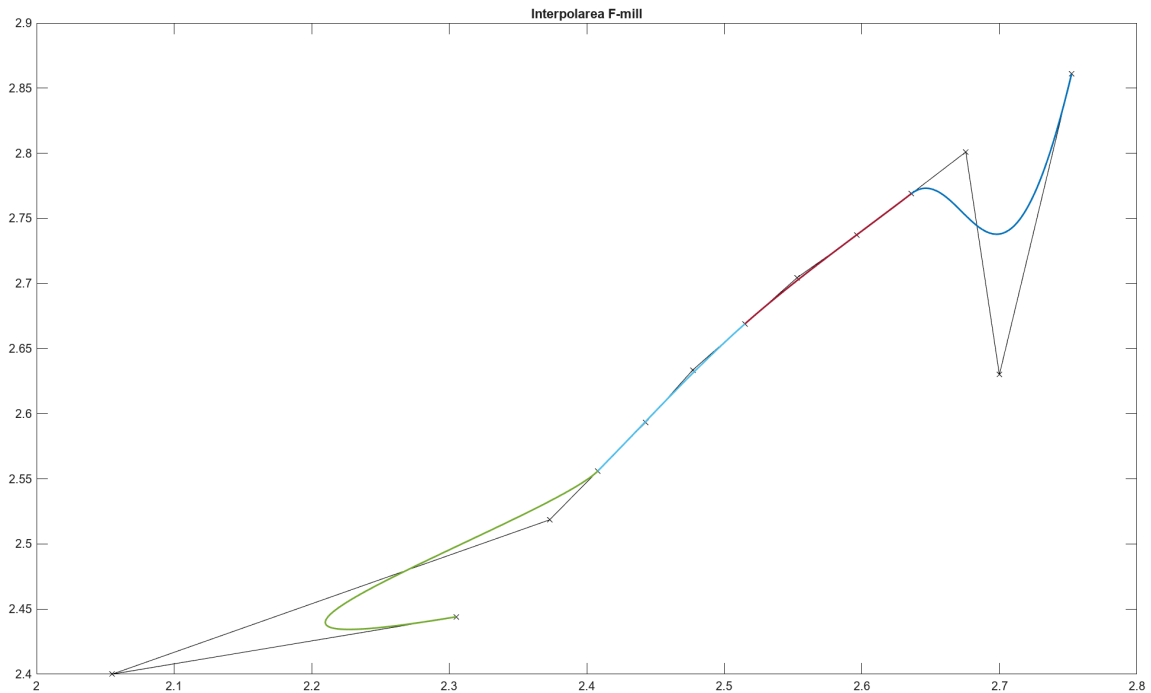
Raportul proiectului de creativitate
Oltean Costin-Petruț

Interpolarea funcției prin metoda spline cubică constă în găsirea unei curbe continue și netede care trece prin punctele de date date. Acest lucru implică împărțirea intervalului dat în segmente, determinarea coeficienților spline-ului cubic pentru fiecare segment și construirea spline-ului cubic în întregul interval. Spline-ul cubic rezultat poate fi apoi utilizat pentru a aproxima valoarea funcției în orice punct din intervalul original. Metoda spline cubică este eficientă și produce o interpolare precisă și netedă a datelor.

```
clear;
figure;
p = [2.305 2.408 2.515 2.636 2.7525; 2.444 2.556 2.669 2.769 2.861];
a = 2.055;
e = 2.4;
c = 2.7;
d = 2.630;
n=max(size(p));
l=zeros(2,n-2);
for j=1:(n-2)
    l(:,j) = p(:,j+2) - p(:,j);
end
ultim=3*(n-1)+1;
b= ones(2,ultim);
b(:,1) = p(:,1);
b(:,2) = [a;e];
b(:,3) = p(:,2)-(1/6)*l(:,1);
for k=1:(n-3)
    b(:,3*k+1) = p(:,k+1);
    b(:,3*k+2) = p(:,k+1) + (1/6) *l(:,k);
    b(:,3*k+3) = p(:,k+2) - (1/6) *l(:,k+1);
end
b(:,3*(n-2)+1) = p(:,n-1);
b(:,3*(n-2)+2) = p(:,n-1) + (1/6) *l(:,n-2);
b(:,3*(n-2)+3) = [c,d];
b(:,ultim) = p(:,n);
i=1;
nr=1;
ng=floor(ultim/4)+1;
while nr<=ng
    plot(b(1,i:i+3), b(2,i:i+3), 'kx-');
    hold on;
    nr=nr+1;
    i=i+3;
end
t=0:.001:1; %parametrul
B0=(1-t).^3;
B1=3.*(1-t).^2.*t;
B2=3.*(1-t).*t.^2;
B3=t.^3;
B=[B0;B1;B2;B3];
m=length(t);
```

Raportul proiectului de creativitate Oltean Costin-Petruț

```
x=zeros(2,m);  
i=1;  
nr=1;  
while nr<=ng  
x=b(:,i:i+3)*B;  
plot(x(1,:),x(2,:), 'LineWidth',1.3);  
nr=nr+1;  
i=i+3;  
end  
for k=1:ng  
end  
hold off  
title('Interpolarea F-mill');
```



Interpolarea Spline:

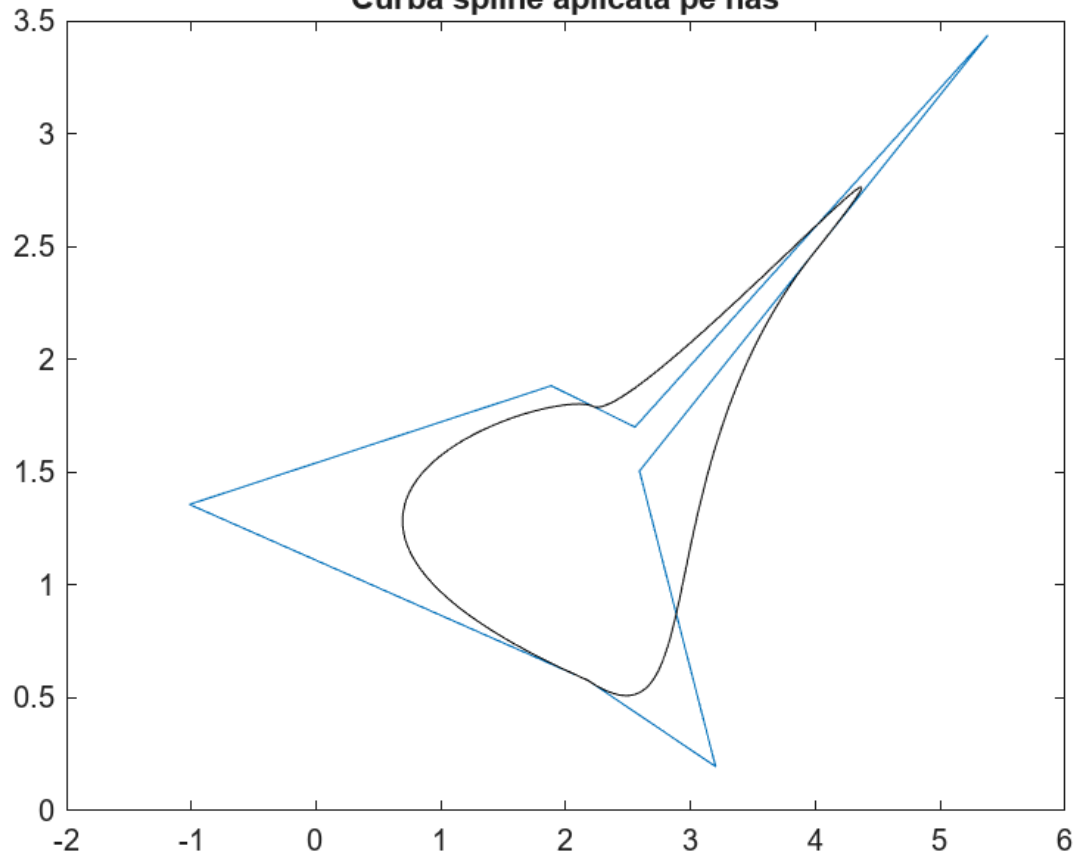
Interpolarea spline cubică Bézier este o metodă care combină proprietățile spline-urilor cubice și curbilor Bézier. Aceasta implică împărțirea intervalului de interpolare în segmente și construirea unui spline cubic Bézier pentru fiecare segment, folosind punctele de control corespunzătoare. Acest spline cubic Bézier rezultat poate fi apoi utilizat pentru a aproxima valorile funcției în interiorul intervalului dat. Această metodă oferă o interpolare precisă și netedă, cu o structură ușor de gestionat și de evaluat.

```
figure;  
p = [2.1754 -1.014 1.883 2.554 5.38 2.59 3.2 2.1754 ; 0.578 1.357 1.883 1.7  
3.436 1.505 0.195 0.578];
```

Raportul proiectului de creativitate
Oltean Costin-Petruț

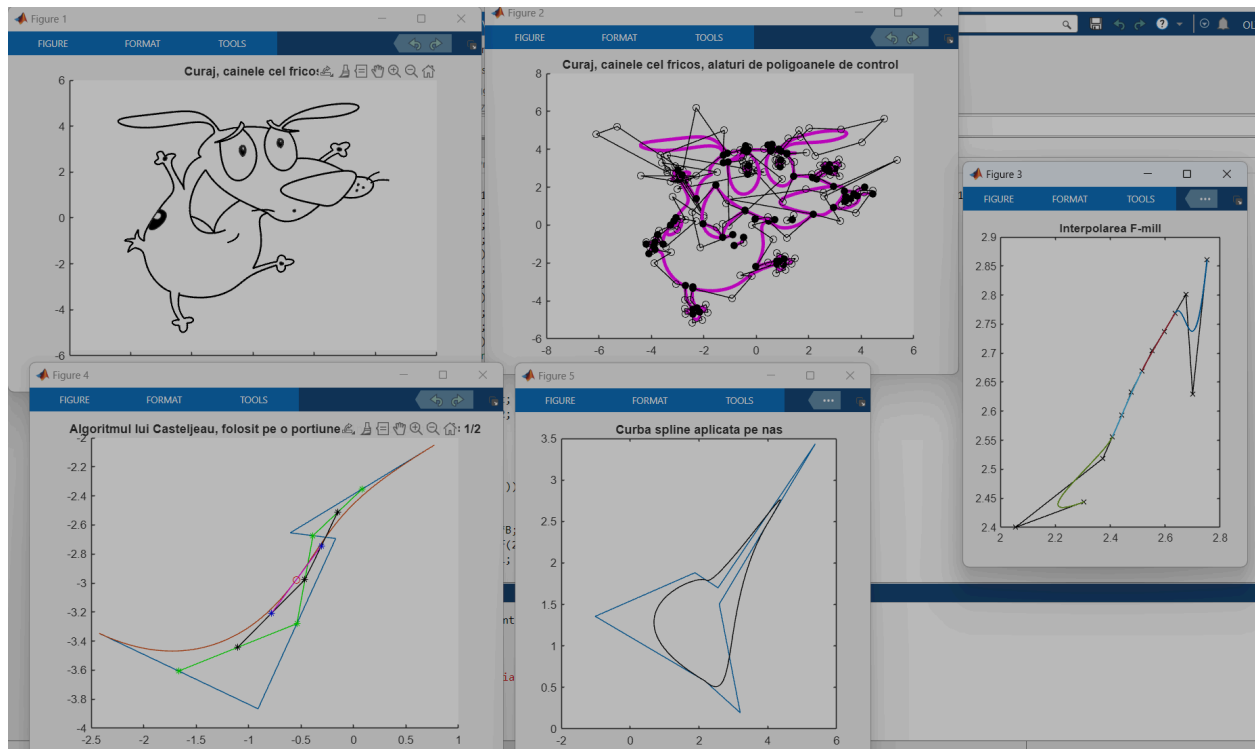
```
b(:,1)=p(:,1);
b(:,2)=p(:,2);
b(:,3)=p(:,3);
b(:,4)=(p(:,3)+p(:,4))/2;
b(:,5)=p(:,4);
b(:,6)=p(:,5);
b(:,7)=(p(:,5)+p(:,6))/2;
b(:,8)=p(:,6);
b(:,9)=p(:,7);
b(:,10)=p(:,8);
%obținerea curbei spline
t=0:0.001:1;
B0=(1-t).^3;
B1=3.*(1-t).^2.*t;
B2=3.*(1-t).*t.^2;
B3=t.^3;
B=[B0;B1;B2;B3];
i=1;
index=1;
plot(p(1,:),p(2,:));
while index<=3
    hold on;
    f=b(:,i:i+3)*B;
    plot(f(1,:),f(2,:), 'k'); %reprezentarea curbei spline
    index=index+1;
    i=i+3;
end
title('Curba spline aplicata pe nas');
```

Curba spline aplicata pe nas



Raportul proiectului de creativitate

Oltean Costin-Petruț



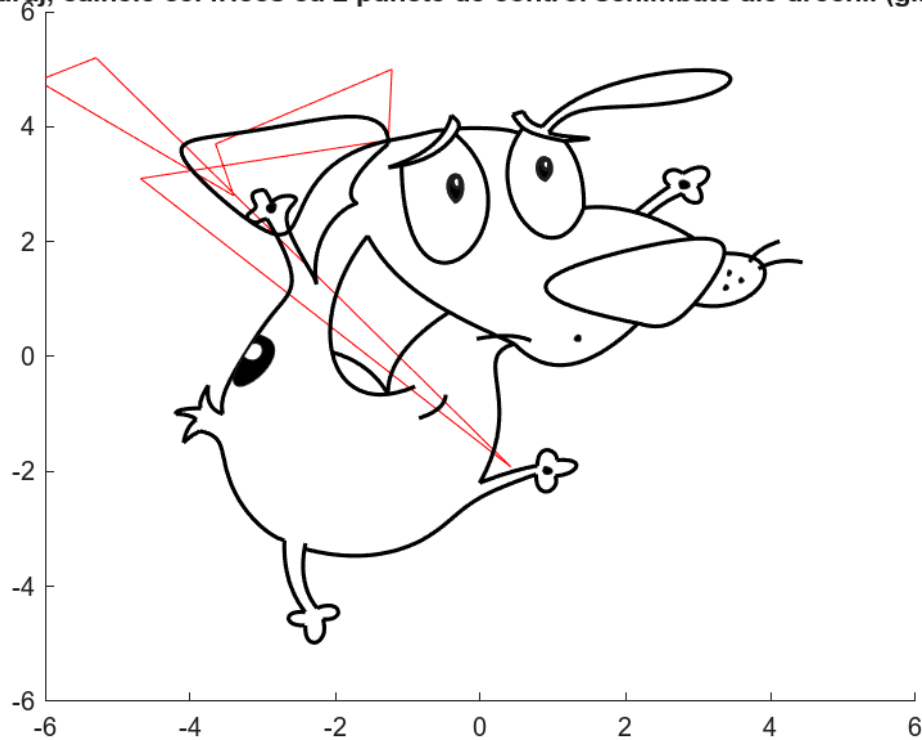
Modificare locală:

```
figure;  
t = 0:0.01:1;  
title('Curaj, cainele cel fricos cu 2 puncte de control schimbate ale  
urechii (ginput)');  
hold on  
axis ([-6 6 -6 6]);  
g = ginput(2);  
g = g';  
b = [-1.1, g(1,1), g(1,2), -5.3, -6.1, -3.4, -3.65, -1.22, -1.27; 3.8,  
g(2,1), g(2,2), 5.2, 4.8, 2.8, 3.7, 5, 3.7];  
f = ReprezentareCurbaBezier(t,b);  
plot(b(1,:),b(2,:), 'r-');  
plot(f(1,:),f(2,:), 'k', 'LineWidth',1.5);  
Contur_dreapta_cap;  
Mana_dreapta;  
Coadă;  
Picior_stanga;  
Picior_dreapta;  
Contur_dreapta;  
Bot_si_nas;  
Contur_stanga_cap;  
Spranceana_stanga_ureche;  
Ochi_dreapta;
```

Raportul proiectului de creativitate
Oltean Costin-Petruț

```
Spranceana_dreapta;  
Ochi_stanga;  
Mana_stanga  
Gura;  
Pete_maini_picioare;  
Interior_ochi;  
Pete_corp;  
Mustati;  
Alunite;
```

Curaj, cainele cel fricos cu 2 puncte de control schimbate ale urechii (ginput)



Grad de îndeplinire al celor 5 cerințe:

Cerința 1 : Complet;
Cerința 2 : Complet;
Cerința 3 : Complet;
Cerința 4 : Complet;
Cerința 5 : Complet;

Bibliografie:

- https://en.wikipedia.org/wiki/De_Casteljau%27s_algorithm
- <https://www.desmos.com/calculator> (pentru aflarea punctelor de control)

Raportul proiectului de creativitate
Oltean Costin-Petruț

- Suport Laborator